

FINAL V2 — MEMBER 5 AMENDMENTS

Gap, Matching, Roadmap + Shortlist Engine

Freeze the baseline formulas

- Gap priority baseline = $\text{normalized_demand} \times \text{trend_multiplier} \times \text{gap_severity} \times \text{role_importance}$.
- Suggested match baseline = $0.60 \text{ skill coverage} + 0.20 \text{ semantic similarity} + 0.10 \text{ experience} + 0.05 \text{ education} + 0.05 \text{ location/work-mode}$; renormalize missing optional components.
- Return matched skills, missing skills, every component score, final score, scoring_version and explanation.

Hiring Manager requirement

- The same matcher must serve applicant→job and recruiter→candidate. Do not build a second recruiter-specific scoring system.
- Shortlist ranking should reuse match outputs, then apply deterministic tie-breakers such as required-skill coverage and experience fit.

Roadmap requirement

- Course/project sequencing must respect prerequisite edges. Validate the DAG and fail safely on cycles.
- Career What-If should call existing gap/match/demand functions twice and should not introduce a new ML model.

Member 5 — Matching, Gap & Recommendation Engine

ML/Recommendation Engineer

Your mission

Own this workstream end-to-end, publish stable contracts, and integrate incrementally. Branch: **feature/gap-matching-roadmap**



1. Exact work you build

- Task 01** — Define profile/target representations using canonical skill IDs, proficiency and evidence.
- Task 02** — Implement F3 gap analyzer with explicit overlap first and demand/trend-weighted priority.
- Task 03** — Add semantic similarity carefully; taxonomy matches must remain authoritative.
- Task 04** — Implement F4 reusable match(profile_a, profile_b) for both applicant→job and recruiter→candidate.
- Task 05** — Implement explainable score components: skill coverage, semantic similarity, experience fit and optional education fit.
- Task 06** — Implement F5 roadmap ranking + prerequisite DAG topological sort.
- Task 07** — Implement recruiter shortlist top-N ranking and tie-break rules.
- Task 08** — Implement Career What-If aggregation and evaluation fixtures.
- Task 09** — Version scoring weights and model outputs.

Figma screens in your scope

Screen	Backend purpose	Your responsibility
Skill Gap	Real data/action must replace mock state.	Implement your module contract and publish response schema.
Roadmap	Real data/action must replace mock state.	Implement your module contract and publish response schema.
Job Matches	Real data/action must replace mock state.	Implement your module contract and publish response schema.
Candidate Matching	Real data/action must replace mock state.	Implement your module contract and publish response schema.
Shortlist	Real data/action must replace mock state.	Implement your module contract and publish response schema.
Career What-If	Real data/action must replace mock state.	Implement your module contract and publish response schema.

2. APIs and database

Area	Method	Endpoint	Role	Purpose	DB	AI
Gap	POST	/api/v1/gaps/analyze	Candidate/Hiring/Gov	Profile vs target	skill_gaps	F3
Roadmap	POST	/api/v1/roadmaps/generate	Candidate	Learning plan	roadmaps,steps	F5
Jobs	GET	/api/v1/jobs	Candidate/Hiring	Search/filter	jobs	F1/F4
Jobs	POST	/api/v1/jobs	Hiring	Create job	jobs,job_skills	F6 optional
Matching	POST	/api/v1/matching/jobs	Candidate	Rank jobs	job_matches	F4
Matching	GET	/api/v1/jobs/{id}/candidates	Hiring	Rank candidates	matches	F4
Shortlist	POST	/api/v1/shortlists	Hiring	Add decision	shortlists	F5

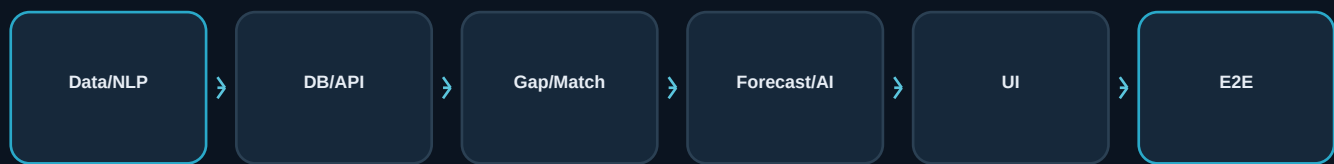
Relevant tables

Table	Purpose	Key columns
skills	Canonical taxonomy	id PK; canonical_name UNIQUE; category; aliases JSONB; source; version
candidate_skills	Candidate proficiency	candidate_id; skill_id; proficiency; evidence; source; UNIQUE pair
roles	Normalized roles	id; name; sector; source
role_skills	Role requirements	role_id; skill_id; importance; min_proficiency
jobs	Job records	id; recruiter_id; title; company; description; district; sector; exp; salary; mode; source; dates; status
job_skills	Job→skill	job_id; skill_id; importance; required_level
skill_demand_history	Demand time series	skill_id; district_id; sector; period; posting_count; demand_score; source
skill_forecasts	Forecast outputs	skill_id; district_id; sector; horizon; forecast; trend; model_version; metrics
courses	Training supply	id; provider; name; district_id; sector; level; duration; mode; source_url
course_skills	Course→skill	course_id; skill_id; coverage_level
course_prerequisites	Course DAG	course_id; prerequisite_course_id; UNIQUE pair
skill_gaps	Computed gaps	candidate_id; target_role_id; skill_id; severity; demand; trend; priority; explanation
roadmaps	Roadmap header	id; candidate_id; target_role_id; generated_at; model_version
roadmap_steps	Ordered plan	roadmap_id; skill_id; course_id; project; step_order; duration; status
job_matches	Candidate/job scores	candidate_id; job_id; score; skill_score; semantic_score; experience_score; matched_json; missing_json
shortlists	Shortlist header	id; job_id; recruiter_id; created_at
shortlist_items	Candidate decisions	shortlist_id; candidate_id; rank; decision; notes; decided_at

3. Dependencies and integration

- Wait for Member 4: canonical skills + demand/trend.
- Wait for Member 3: service/API interface.
- Wait for Member 6: forecast signal.
- Provide Members 1/2: score explanations and acceptance examples.

Integration order



4. Testing checklist

- Unit tests cover every scoring/parser/transform function or critical UI behavior owned.
- At least one negative case exists for invalid/missing input.
- Critical outputs are deterministic when the seeded demo data is used.
- Every score includes explanation + version metadata.
- No secret/API key is in source control.
- No screen relies on mock business data after its API is connected.
- A clean database reset reproduces the expected demo.
- The module starts from the README commands.

Definition of Done

- Figma functionality in your scope is reachable through real routes/actions.
- Backend, DB and AI dependencies are connected through documented contracts.
- Loading, empty, error and permission states are implemented.
- Tests pass and the branch merges cleanly into develop.
- Demo seed produces the expected screenshots/values.
- PR includes API/schema/model/data notes and any known limitation.

5. 7-day execution schedule

Day	Output
1	Representations + formulas
2	Gap baseline
3	Semantic similarity + explanations
4	Job matching + candidate ranking
5	Roadmap + prerequisites
6	What-if + evaluation
7	API integration + demo tuning

Final rule

Do not wait for all modules. Integrate a thin vertical slice as soon as upstream contracts exist.